CAMPUS RESOURCE ALLOCATION SYSTEM

Person A — Deadlines 2 and 3

The authentication column and the authorization boundary: every file, what it does, and why it is written that way.

Owner Person A (correctness core) **Deadline 2** Session 7 · commit `25f29cc` **Deadline 3** Session 8 · commit `fa93526`
Status both MET, re-verified 2026-08-23

1 Where these two deadlines sit

The project splits work by **layer and verification**, not by feature. The flagship GPU transaction is a single ~60-line function and cannot be halved, so **A builds the correctness core** — security, auth, authorization, quotas, idempotency, the GPU transaction, waitlist promotion — while **B builds the resources and the tests designed to break A's core**. Each person's benchmark is the proof of the other's work.

DEADLINE	A'S COLUMN	OUTCOME
2 — Auth vs. Read Paths	Argon2 hashing, JWT encode/decode, role as a token claim, `POST /auth/register`, `POST /auth/login`, duplicate email → 409 not 500	MET — checkpoint *"login returns a token containing a role"*
3 — Authorization vs. Rooms	`require_role` dependency, real `get_current_user`, delete the stub, `GET /me` — plus two ADMIN routes the plan never assigned to anyone	MET — checkpoint *"student token on POST /gpus returns 403"*

> **The shape of the whole design, stated once.** Authentication and authorization are **boundary** concerns — they can be decided from identity alone, so they live in dependencies and raise HTTP errors freely. Capacity, quota and exactly-once are **invariants** — they can only be decided under a lock, so they live inside a transaction. Every file below sits on one side of that line, and knowing which side it is on explains most of its content.

> **Provenance note.** A's Deadline 3 work (session 8) was committed inside `fa93526`, which carries the git author of Person B because sessions 8 and 9 ran back-to-back in one sitting and were committed together. Git blame on `core/dependencies.py` will therefore attribute A's code to B. Attribution comes from `WORK_LOG.md`, not from `git log`.

2 Deadline 2 — files changed

Commit `25f29cc`, 14 files, +1023 lines. Written strictly in dependency order — `security.py` → `schemas.py` → `service.py` → `router.py` — so each file was fully testable before the next one existed.

FILE	LINES	WHAT IT IS
`app/core/security.py`	105 new	Argon2 hashing + JWT encode/decode. Zero HTTP knowledge.
`app/auth/schemas.py`	52 new	Request/response models: register payload, user read model, token envelope.
`app/auth/service.py`	89 new	Domain half: `register()`, `authenticate()`. Raises exceptions, not status codes.
`app/auth/router.py`	80 new	HTTP half: the two public routes, and the mapping from exception to status code.
`app/core/errors.py`	34 new	**Not in the plan.** The JSON envelope every coded error arrives in.
`app/main.py`	+7	Mounts `auth_router` under `API_PREFIX`.
`scripts/check_auth.py`	235 new	25-assertion gate; exits non-zero, so it is a gate and not a paragraph.
`DECISIONS.md`	+168	The reasoning record — the interview cheat-sheet.
`ARCHITECTURE_AND_WORKFLOWS.md`	+41	Error envelope section; Workflow A corrected for form-encoding and string `sub`.
`WORK_LOG.md / EXECUTION_PLAN.md`	+205	Session 7 entry; Deadline 2 marked MET with its honest qualification.

3 Deadline 2 — file by file

`app/core/security.py` WRITTEN FIRST

What it is. Four functions and one module constant: `hash_password`, `verify_password`, `create_access_token`, `decode_access_token`, and `DUMMY_PASSWORD_HASH`.

Why it is written first. It imports only `app.core.config` — no models, no database session, no FastAPI. That makes it the only part of the auth column fully testable without a database, and it means nothing in it can grow a hidden dependency on a request. Everything above it in the stack can be built against a file that is already known good.

Why it raises no HTTP errors. `decode_access_token` lets PyJWT's `InvalidTokenError` propagate. Turning that into a 401 is the boundary layer's job at Deadline 3. A service that knows about status codes is a service that cannot be called from anywhere else — from a script, a test, or a background job.

LINES	DETAIL WORTH KNOWING
31	**`DUMMY_PASSWORD_HASH` is public on purpose.** `authenticate()` verifies against it when the email does not exist, so a login attempt on an unknown address costs the same ~50 ms as one on a real address.
34–43	Argon2id, with cost parameters encoded into the hash string itself. This is what let B's seed script hash three users with a bare `PasswordHasher()` *before this file existed* and still have them verify.
46–61	**Catches `(VerificationError, InvalidHashError)`, never raises.** See §4 — the exception hierarchy here is not the one anyone would guess.
64–89	HS256. `"sub": str(user_id)` at line 84 is load-bearing; `role` travels as a claim at line 85.
92–105	`exp` is checked by the library, not by us. `InvalidTokenError` is the common base of expiry, bad signature and malformed subject — so the caller's 401 handler is a single `except` clause.

> **Why `sub` is stringified.** PyJWT ≥ 2.10 requires a string subject but **enforces it on decode, not encode**. An integer `sub` produces a perfectly valid-looking token, and then raises `InvalidSubjectError` on every subsequent request. The symptom would appear in `get_current_user` while the bug lived in `create_access_token` — a failure that points at the wrong file. This was found by `scripts/check_jwt.py` in session 4, before any code depended on it.

> **Why the role travels in the token at all.** So a future stateless service could authorise without a database round-trip, and because Deadline 2's checkpoint is literally *"login returns a token containing a role"* — the claim is the demonstrable half of the auth story. Deadline 3 later established that this system does **not** authorise on it (§8).

app/auth/schemas.py

What it is. Three Pydantic models — `RegisterRequest`, `UserRead`, `TokenResponse`.

- **`email` is a constrained `str`, not `EmailStr`** (lines 8–23). `EmailStr` pulls in `email-validator`, a dependency the image does not have — and adding one forces a rebuild on both machines for a format check that guarantees nothing the system relies on. What the system relies on is `UNIQUE(email)` in the database, which no client-side validation can substitute for.
- **`UserRead` has no `password_hash` field** (lines 26–38). The property that matters most about this model is a *negative* one: no handler can leak the hash by returning an ORM object, because the field does not exist to be serialised. `check_auth.py` asserts this by listing the response keys, which proves the model — not the ORM object — is what gets serialised.
- **The role is deliberately *not* duplicated into `TokenResponse`** (lines 41–52). It is a claim inside the token, which is the copy authorization reads. A second copy in the envelope would be a number a client could trust that the server never checks.

app/auth/service.py

What it is. The domain half. Raises `EmailAlreadyRegistered`, not a 409; returns `None` for bad credentials, not a 401. The router maps both. This is what lets Deadline 4's transactions be tested by calling services directly rather than through HTTP.

REGISTER() — LINES 26–66

There is deliberately **no `SELECT ... WHERE email = ?` before the INSERT**, and its absence is the entire point. Two simultaneous registrations of one address can both run that check, both see nothing, and both proceed — so the check would pass exactly when it matters least, while making the code *look* like the race was handled. `UNIQUE(email)` is what makes the second insert impossible, so **the INSERT is the check**, and catching `IntegrityError` is how its result is read.

> **The trap inside the trap.** `db.rollback()` must be the *first* statement in the `except` block (line 56). After an `IntegrityError` the session is in a failed state, and any further statement on it raises `PendingRollbackError` — which would surface as a 500 and bury the 409 it was supposed to produce.

`db.refresh(user)` at line 65 is also deliberate. `expire_on_commit=False` keeps the object readable after commit, but `id` and `created_at` are server-generated and `created_at` was never loaded — so serialising `UserRead` would emit a lazy SELECT from inside the response layer. The session settings exist to stop statements appearing at surprising points; relying on one here would spend that.

AUTHENTICATE() — LINES 69–89

Does not distinguish "no such email" from "wrong password" — both are one 401, and telling them apart would confirm which addresses exist. On the miss path it still runs a full Argon2 verify against `DUMMY_PASSWORD_HASH` (line 83), because **an identical response body delivered at a distinguishable response time is still a disclosure**.

`app/auth/router.py`

What it is. Two public routes — the only two that remain reachable without a token after Deadline 3.

LINES	DETAIL
19–44	`POST /register` → 201. `EmailAlreadyRegistered` becomes `coded_error(409, "EMAIL_ALREADY_REGISTERED", …)`.
24–34	Docstring records an **accepted hole**: `role` comes from the request body, so a caller can register as ADMIN. See §4.
47–80	`POST /login`, **form-encoded not JSON**, via `OAuth2PasswordRequestForm`.
66	The `username` → email mapping happens here, exactly once. The rest of the system says `email` everywhere.
72–76	401 carries `WWW-Authenticate: Bearer` — what makes it a spec-conforming 401 rather than a 401-shaped 403.

`app/core/errors.py` NOT IN THE PLAN

Why this file exists at all. Deadline 1 agreed the error *codes* — `CAPACITY_EXHAUSTED`, `QUOTA_EXCEEDED`, `IDEMPOTENCY_KEY_REUSED` — but never the JSON they arrive in. Nobody noticed, because no endpoint had returned one yet. Duplicate registration is the first, so the shape was settled here rather than improvised three more times at Deadlines 4 and 5.

```
{"detail": {"code": "QUOTA_EXCEEDED", "message": "human-readable prose"}}
```

Why an envelope at all. `CAPACITY_EXHAUSTED` and `QUOTA_EXCEEDED` are *both* 409 and the caller's remedy is opposite: wait or try another cluster, versus release something you already hold. A client cannot tell them apart from the status line and must not have to parse English to do it. `code` is the contract; `message` may be reworded freely. B's benchmarks assert on `code`.

> **Why it returns rather than raises.** Call sites read `raise coded_error(...)`, so the `raise` stays visible at the point of failure instead of being hidden inside a helper — which matters most when the failure is one branch of a transaction.

4 Deadline 2 — the decisions, and why

4.1 `InvalidHashError` is a `ValueError`, not an `Argon2Error`

Verified in the container rather than assumed, and it changed the code:

```
VerifyMismatchError -> VerificationError -> Argon2Error -> Exception
InvalidHashError    -> ValueError        -> Exception
```

The intuitive single clause, `except Argon2Error`, catches a wrong password but **not a malformed or empty `password_hash`** — which would escape the login handler as a 500. `verify_password` therefore catches `(VerificationError, InvalidHashError)` and returns `False` for both, because "this stored hash is garbage" and "this password is wrong" are the same answer to a caller: these credentials do not authenticate anyone.

4.2 Login is form-encoded; register is JSON

The one genuinely arguable call in the column, so both sides are recorded.

FOR	AGAINST
`INIT_PLAN.md` §4 names the OAuth2 password flow, and `python-multipart` has sat in `requirements.txt` since Deadline 1 doing nothing else. Following the flow gives `/docs` a working **Authorize** button the moment Deadline 3 registers the bearer scheme — the difference between demoing RBAC by clicking and demoing it by pasting headers into every request. That is worth real money at Deadline 10.	The email arrives in a field the spec insists on calling `username`, and B's harness must send `data=` rather than `json=` for this one route.

Chosen **for**, with the mitigation that the mapping happens once, in the handler. That Authorize button was cashed at Deadline 3 and is now asserted by `check_rbac.py`.

4.3 Both 401s are identical, including their timing

That the bodies match is obvious. Less obvious: without help they arrive at very different *speeds* — an unknown address skips Argon2 entirely and answers in microseconds, while a real one pays ~50 ms for a full verify. That difference is measurable from outside over a handful of requests, and turns the login endpoint into an oracle for which addresses are registered.

4.4 Two holes accepted deliberately, and recorded rather than shipped quietly

- **A caller may register themselves as ADMIN.** `role` comes from the request body, as Workflow A specifies. The alternative is bootstrapping the first admin out of band, which adds a seeding concern to every clean-room run — for a project whose claim is about concurrency, not identity management. `scripts/seed.py` already supplies three demo accounts. Belongs in the README's limitations, where a reader meets it as a decision rather than discovering it as an oversight.
- **`email` is not format-validated.** See §3. The guarantee lives in the database.

4.5 The audit that caught a worthless test

Prompted by re-checking Deadline 2 against the plan line by line rather than taking the log at its word. The first version of the duplicate-email check registered one address **twice in a row** and asserted 409. It passed. It was close to worthless.

> The plan does not ask for a 409. It asks for *"409, not 500: catch the IntegrityError, **because** two simultaneous registrations can both pass a 'does this email exist?' check."* A sequential retry is **exactly the case a naive pre-flight SELECT handles correctly** — so the test agreed with the implementation the project had deliberately rejected, and would have gone on agreeing with it forever.
>
> **The general rule, which now governs every benchmark ahead:** before writing an assertion, ask *which implementation would fail it*. If the answer is "none of the ones we were choosing between", it is measuring something else.

Replaced with 8 registrations of one address released together on a `threading.Barrier`: `1 × 201, 7 × 409`, zero 500s, and — the assertion that actually matters — **one row**, counted in the database rather than inferred from status codes. Status codes are what the server claimed; the row count is what happened.

5 Deadline 3 — files changed

A's half of commit `fa93526`. The headline is a deletion: the Deadline 1 stub is gone, and **zero edits were needed at B's ten call sites**.

FILE	CHANGE	WHAT IT IS
`app/core/dependencies.py`	+143 rewrite	Real JWT decode, real role enforcement. Was a hardcoded-ADMIN stub.
`app/users/router.py`	40 new	`GET /me`. New package; no service file, on purpose.
`app/gpus/router.py`	+34	`POST /gpus` [ADMIN] — the route the checkpoint names.
`app/gpus/service.py`	+20	`create_cluster()`, allocated forced to 0.
`app/gpus/schemas.py`	+16	`GPUClusterCreate` — deliberately has no `allocated` field.
`app/rooms/router.py`	+ADMIN route	`POST /rooms` [ADMIN], same gate. *(Rest of this file is B's.)*
`app/core/config.py`	+8	`API_PREFIX` moved here from `main.py` to break an import cycle.
`app/main.py`	+29 / −6	Mounts `users_router`; imports the prefix from config.
`scripts/check_rbac.py`	331 new	34-assertion gate. Opens with six ways of arriving without a valid identity.

6 Deadline 3 — file by file

`app/core/dependencies.py` — the stub deleted

THE DEADLINE

What it was. A 39-line stub returning a hardcoded ADMIN, shipped at Deadline 1 so B could build routes before real auth existed. It took no parameters and read no header.

What it is now. The API boundary: decode → 401, load the user → 401 if gone, compare the role → 403 — all before any handler body runs.

LINES	DETAIL
39	`OAuth2PasswordBearer(tokenUrl=…)` registers the bearer scheme. **No leading slash**, so Swagger resolves it relative to the docs page and the Authorize button survives the app being mounted under a sub-path.
42–55	`_unauthorized()` — 401 with `WWW-Authenticate: Bearer`, uncoded on purpose (§7.4).
75–78	One `except jwt.InvalidTokenError` covers expiry, bad signature *and* malformed subject, because it is their common base. This single clause is the reason `security.py` was allowed to raise no HTTP errors of its own.
85–88	`int(claims["sub"])` — the other half of the PyJWT trap. `str()` on the way out, `int()` on the way in.
90–95	A correctly signed token naming a user who no longer exists → 401. Rare, but this is the one place it can be caught; returning `None` would push a `NoneType` error into every handler downstream.
127–133	`require_role`'s inner checker, taking `user = Depends(get_current_user)` — a **dependency**, not an `if`.

> **The frozen-interface payoff, measured.** Deadline 1 froze the import path, the call shape and the return type — not the parameter list, since the real version obviously needs a token and a session that the stub did not. The swap therefore cost **zero edits** across B's ten call sites in `gpus/`, `rooms/` and `courses/`. This is the single clearest return on a Deadline 1 decision anywhere in the project.

`app/users/router.py` — `GET /me`

Why it belongs to this deadline and no other. It is the end-to-end proof that a real token decodes to a real user carrying a real role — the entire claim Deadline 3 makes. A 403 alone cannot make that claim, because *a route that rejects everyone also produces 403s*.

Why it was homeless until session 5. It appeared in `INIT_PLAN.md` §12 and `ARCHITECTURE_AND_WORKFLOWS.md` with **no deadline assigned** — the same gap that left the admin PATCH endpoints and `GET /me/quota` ownerless.

Why there is no `users/service.py`. `INIT_PLAN.md` §15 splits routers from services because the domain half is the part worth testing without a client. Here the domain half is empty — `get_current_user` has already produced the row, so a service function would be `def get_me(user): return user`. A file that only forwards is a file to keep in sync for nothing. `GET /me/quota` lands in this module at Deadline 6 and *will* bring a service, because reading quota policy and SUMming held units is real domain logic.

Why it reuses `auth.schemas.UserRead`. A second model of the same row would drift, and the property that matters most about that model is the negative one — no `password_hash` field. A guarantee like that is worth having in exactly one place.

`app/gpus/*` and `app/rooms/router.py` — `POST /gpus`, `POST /rooms` UNASSIGNED IN THE PLAN

Why A wrote routes that were not in A's column. Deadline 3's checkpoint is *"student token on `POST /gpus` returns 403"*. **That route did not exist.** Neither did any other admin-only route — so `require_role` was about to ship with nothing guarding anything, and the checkpoint could not have been run at all.

Creating a resource is `[ADMIN]` in the role matrix of both `INIT_PLAN.md` §13 and `ARCHITECTURE_AND_WORKFLOWS.md` §8, and appears in the API list of §12 — with no deadline assigned. **Third instance of that exact gap**, after `GET /me` and the admin PATCH endpoints.

> **Pattern now named:** an endpoint that appears in a specification but in no deadline's column belongs to nobody, and is discovered by whichever checkpoint trips over it. A fourth instance is already known and still unassigned — `POST /courses` and `POST /offerings`.

`GPUClusterCreate` has no `allocated` field. It is derived state owned by the allocation transaction, and letting a caller set it would allow a cluster to be born already over-allocated — the precise invariant `gpu_capacity_sane` exists to protect. New clusters start at zero, always.

`RoomCreate` carries `capacity`, and it is a different kind of number — seats in a room, a physical fact. Rooms are allocated by *interval*, not by unit, so there is no counter to guard on write. Worth stating because two resources sharing one base table invites the assumption that they share an allocation model, and they deliberately do not.

`app/core/config.py` — `API_PREFIX` relocated

`OAuth2PasswordBearer(tokenUrl=…)` needs the prefix, and importing `main.py` from `core/dependencies.py` is a cycle — `main.py` imports the routers, which import the dependency. The value and its reasoning are unchanged from session 6; only its address moved, so there is still exactly one place to change it.

7 Deadline 3 — the decisions, and why

7.1 What Deadline 2's green checkpoint was actually worth

`GET /gpus` returned 200 with a bearer token — **and also with no token at all**, because the stub read no header. Session 7 recorded that honestly at the time rather than letting the green result stand for more than it was.

> A route that answers the same way with and without a credential has tested the happy path of **routing**, not of **authentication**.

`check_rbac.py` opens with the negative case for that reason: six ways of arriving without a valid identity — absent, malformed, wrongly signed, expired, integer `sub`, and a validly signed token naming a user who does not exist — each asserted to be 401. **Every one of them returned 200 against the previous day's build.**

7.2 The 403 must fire before the handler body, and a status code cannot show that

`require_role` is a dependency, not an `if` at the top of a handler. FastAPI resolves the dependency chain before calling the function, so a rejected caller cannot leave partial state behind.

An `if user.role != ADMIN: raise` written as the *first* line of the handler behaves identically from outside — same 403, same body — and the two only diverge when that check drifts one line below the INSERT, which is exactly the kind

of edit nobody notices in review. So the assertion is **not the status code**; it is the **row count on both sides of the 403**, read from the database.

7.3 The dependency chain has an observable order

```
authentication (401)  ->  authorization (403)  ->  validation (422)
```

A student sending a body that is *also* invalid gets **403, not 422**. A 422 there would mean the request body was parsed and validated for a caller with no business reaching the endpoint. Asserted directly in `check_rbac.py`.

7.4 401 and 403 stay uncoded

`coded_error()` exists for failures a client must *branch* on. There is exactly one remedy for a 401 (get a token) and one for a 403 (stop), so both keep FastAPI's plain string `detail` and the envelope stays reserved for cases where the code carries information the status line does not. Two smaller calls in the same spirit:

- **Every 401 is byte-identical**, whatever went wrong. Which one it was is not the caller's business, and distinguishing them hands an attacker a probe.
- **The 403 does not name the required role.** That is policy information, and a caller cannot act on it.

8 The reversal: the role is read from the database, not the claim

Deadline 1 decided the opposite. The reversal is worth stating plainly, because the original reasoning **was not wrong** — it was overtaken by another decision made in the same session.

The Deadline 1 claim was: put `role` in the token so `require_role` authorises with no database round-trip, accepting that a role change lags until the token expires. The problem is that the *other* Deadline 1 decision — freezing `get_current_user() -> User` — means the user row is loaded on every authenticated request anyway. `require_role` depends on `get_current_user`, so by the time it runs the row is already in hand. **The saving was spent before the function that was supposed to collect it was ever reached.**

COPY OF THE ROLE	PROPERTIES
`token claim`	signed, tamper-proof, up to 60 minutes stale
`database row`	already loaded, never stale

With both in hand the fresher one wins, so a demoted admin loses admin on their next request rather than at expiry, and the "accepted tradeoff" recorded at Deadline 1 is simply void — there is no cost being paid for it any more. The claim still earns its place in two ways that are *not* authorization: it is the demonstrable half of the auth story, and it is what a future stateless service would read.

> **The assertion that proves it — and which implementation fails it.** Mint a token signed with the *real* secret, `sub` = the seeded STUDENT's id, `role` claim = `ADMIN`, and POST it at `/gpus`. A build that trusts the claim returns 201 and creates a cluster. This one returns 403. **No forgery is involved** — only this system could have signed that token — which is exactly what makes it a test of *which copy is consulted* rather than a test of the signature.

9 Three recurring lessons

I. The exception hierarchy you assume is not the one the library has. Three instances now: `op.drop_table` does not drop the types `sa.Enum` created (session 2, broke the migration chain); PyJWT enforces a string `sub` at decode and not encode (session 4); `InvalidHashError` is a `ValueError` and not an `Argon2Error` (session 7). Each was found by **running it**, and each would have shipped as a 500 or a broken chain if read off a docstring.

II. A test that the broken implementation also passes is not a test of that requirement. Found in Deadline 2's own audit (§4.5). Governs Deadlines 4, 5 and 7 — Benchmark 2 in particular is only meaningful because the *correct* resource-lock implementation fails it.

III. Write the check as a script, not a shell paste. `check_jwt.py` caught a stale container image on its first run on a second machine. `check_auth.py` caught a requirement tested against the wrong implementation. Neither would have been caught by a one-off paste, which confirms a belief and then evaporates. Four gates now exist and all four are re-run as regressions every session.

10 Verification, reproduced independently

Not taken from the log — re-run against the live stack on **2026-08-23**, at HEAD `fa93526` with a clean working tree. The source is bind-mounted (`.:/code`), so the running code is the committed code; and `check_jwt.py` confirms PyJWT 2.10.1 with `python-jose` absent *inside the container*, ruling out the stale-image failure that has bitten this project twice.

102	4 / 4	0	18
ASSERTIONS	GATES GREEN	FAILURES	ROUTES LIVE

GATE	RESULT	COVERS
`scripts/check_jwt.py`	9/9 · exit 0	The PyJWT swap; the int-`sub` trap at encode vs. decode.
`scripts/check_auth.py`	25/25 · exit 0	Register, duplicate 409 + code, the 8-thread barrier race, validation, login, claims, both 401s, seeded accounts.
`scripts/check_rbac.py`	34/34 · exit 0	Six 401 paths, `GET /me`, the 403 checkpoint, row counts across the 403, the forged-claim test, the OAuth2 scheme in `openapi.json`.
`scripts/check_rooms.py`	34/34 · exit 0	B's column — included because a regression there would also indict the boundary.
`alembic current / check`	head, no drift	`1ca8b85b7626`; "No new upgrade operations detected."
`row counts, 12 tables`	post-seed	Every gate cleans up after itself; no test residue anywhere.

THE ASSERTIONS THAT CARRY THE TWO CHECKPOINTS

```
THE CHECKPOINT: student -> 403 on POST /gpus            -> 403
403 left no cluster behind (fired before the handler)   -> 2 -> 2
403 left no room behind                                 -> 2 -> 2
forged ADMIN claim on a STUDENT row -> 403              -> 403
the forged claim created nothing                        -> 2

8 simultaneous duplicates -> exactly one 201  -> {(201,None):1, (409,'EMAIL_ALREADY_REGISTERED'):7}
race created exactly one row                  -> 1 rows
```

11 What A owns next

Deadline 4 — the flagship, and the heaviest deadline in the plan. A's column is the `quotas/` helper and the GPU transaction:

```
LOCK user row      -> SUM held units -> quota check
LOCK cluster row   -> status check   -> capacity check -> write
lock order recorded in a code comment
```

STATUS	ITEM
Ready	`RoleQuota` and `GPUReservation` models exist, quota rows are seeded, and `ix_gpu_reservations_user_status` is already in place. **No Alembic revision needed.**
Ready	Item 6 was ratified at Deadline 3, so the GPU `BLOCKED` gate has settled semantics — reuse `RESOURCE_BLOCKED` via `coded_error()`.
Ready	A working template: B's `rooms/service.py` reserve path. Same shape — lock, gate on the locked row, no pre-flight SELECT, rollback-first, coded errors out. **One difference:** the GPU path *writes* `allocated`, so it takes `FOR UPDATE` where rooms took `FOR SHARE`.
Decide first	**Outstanding item 9.** The global lock order and the Deadline 7 waitlist-promotion design contradict each other, and Deadline 4 asks A to write the canonical lock order down *in a code comment*. Settle it while writing that comment, not at Deadline 7.
Watch	`(FACULTY, COURSE)` is **absent, not NULL**. `max_units = NULL` means unlimited; no row means the pair is unreachable behind a 403. The quota helper must not conflate them.
Not yet	Idempotency is Deadline 5 — the endpoint ships without the key now, and the key wires in as step (1) of the same transaction later.
Blocked, B's side	Outstanding item 8: `DELETE /reservations/{id}` names a row in two tables with separate id sequences. Joint call, needed before B writes it.

ONE STALE LINE TO FIX IN PASSING

`app/gpus/router.py` line 74 still reads *"outstanding item 6, still unratified"*. It was ratified in session 9, and the room path already carries three comments saying so. The behaviour is correct either way — only the justification is out of date — but the GPU transaction lands in that same module.

Campus Resource Allocation System · Person A, Deadlines 2 and 3 · Sources: `DECISIONS.md`, `WORK_LOG.md` sessions 7–8, `EXECUTION_PLAN.md`, and the source at `fa93526`. All verification figures re-run 2026-08-23, not quoted from the log.